



Universidade de Brasília

Integrantes do grupo: VICTOR MONTIBELLO LOUREIRO (241004630)

SAIMO NASCIMENTO DOS REIS (222012783)

PATRÍCIA TAYNA PEREIRA SILVA (241004603)

LUCAHS UBIRATÃ MENDES DA SILVA FERREIRA JANSE (241036465)

Professor: Fernando Albuquerque

Matéria: Técnicas De Programação (CIC0197)

RELATÓRIO DE DESENVOLVIMENTO DE ARTEFATOS ***SISTEMA DE GERENCIAMENTO DE PROJETOS SCRUM***

1. PARADIGMA DE DESENVOLVIMENTO E TECNOLOGIAS

- **Paradigma de Orientação a Objetos (POO):** O sistema foi inteiramente projetado sob o paradigma de POO em C++, aplicando de forma rigorosa conceitos fundamentais como:
 - *Encapsulamento:* Evidenciado nas classes de domínio (como Texto, Código, Senha), onde os atributos são privados e só podem ser modificados após validação interna nos métodos set.
 - *Abstração e Polimorfismo:* Utilizados na separação de módulos por meio de classes abstratas contendo métodos virtuais puros, definindo contratos operacionais entre a apresentação e os serviços.
- **Persistência com SQLite:** Para garantir que os dados de Projetos, Sprints e Histórias de Usuário não sejam perdidos ao encerrar o programa, a Camada de Serviço integra-se ao banco de dados relacional **SQLite** via biblioteca local. O mapeamento transforma os objetos da POO em registros relacionais dentro de tabelas SQL de forma estruturada.

2. ARQUITETURA EM CAMADAS E RESPONSABILIDADES

O software divide-se em componentes isolados para garantir manutenibilidade e compilação modular limpa no ambiente Code::Blocks:

Camada de Apresentação: Gerencia o fluxo de telas no terminal, menus de opções e interação de entrada/saída via `cin` e `cout`. Ela captura os dados brutos e os converte em objetos de domínio.

Camada de Interfaces (Contratos): Classes abstratas que desacoplam a exibição visual da lógica interna do sistema, blindando o código contra acoplamento excessivo.

Camada de Serviço: Implementa os métodos descritos nas interfaces, efetuando o processamento das histórias de usuário, controle de papéis Scrum e gerenciamento de transações com a persistência.

Camada de Domínios e Entidades: Classes base que encapsulam as regras de formatação (Domínios) e agregam esses dados para representar entidades complexas do modelo de negócios (como a entidade Pessoa ou Projeto).

3. ESTRUTURAÇÃO DOS TESTES E CONVENÇÃO DE CÓDIGO

- **Padrão de Codificação (*Coding Standard*):** O repositório segue convenções estritas de nomenclatura, organização de arquivos cabeçalho (.hpp) e implementação (.cpp), além de tratamento centralizado de erros por meio de lançamento e captura de exceções padrão do C++ (`std::invalid_argument`).
- **Isolamento de Testes:** Conforme as boas práticas de engenharia de software, os códigos de validação de unidade foram desacoplados das classes principais do sistema, permitindo testes de bancada rápidos antes do procedimento de integração e do *smoke test* final.

4. RESUMO GERAL DO SISTEMA COM BASE NO VÍDEO

Voltado para o gerenciamento de projetos que seguem a metodologia Scrum. Durante a apresentação, são exibidas as principais funcionalidades do sistema, demonstrando o funcionamento dos cadastros, consultas, atualizações e exclusões de dados, além da associação entre os diferentes elementos do projeto.

A interface do sistema é baseada em menus no terminal, permitindo ao usuário navegar entre os módulos de gerenciamento de pessoas, projetos, histórias de usuário e planos de sprint. Cada menu apresenta opções específicas, como cadastrar, listar, consultar, atualizar e excluir registros, proporcionando uma navegação organizada e intuitiva. A interação ocorre por meio da entrada de dados pelo teclado e da exibição das informações na tela.

No gerenciamento de pessoas, foi demonstrado o cadastro de usuários com diferentes papéis dentro da metodologia Scrum, como Desenvolvedor, Scrum Master e Product Owner. Também foram apresentadas funcionalidades para consulta por e-mail, alteração de informações, listagem de todos os usuários e exclusão de registros.

Na parte de gerenciamento de projetos, foi criado um projeto contendo código identificador, datas de início e término e o vínculo com um Scrum Master. O sistema também

permite consultar projetos individualmente, listar todos os projetos cadastrados e identificar quais usuários estão associados a cada projeto.

O módulo de histórias de usuário possibilita o cadastro de funcionalidades contendo título, descrição, prioridade, estimativa de desenvolvimento, estado da tarefa e associação ao projeto correspondente. Além disso, foi demonstrada a vinculação de um desenvolvedor responsável pela implementação da história.

Por fim, o gerenciamento de sprint permite cadastrar planos de sprint, definir objetivos, capacidade de trabalho, associar histórias de usuário à sprint, remover essas associações e excluir o plano quando necessário. Dessa forma, é possível acompanhar a organização das atividades previstas para cada ciclo de desenvolvimento.

Em relação aos containers utilizados no projeto, observa-se que o sistema realiza o armazenamento e gerenciamento de coleções de objetos, como listas de pessoas, projetos, histórias e sprints. Esses containers são responsáveis por manter os dados em memória durante a execução do programa, permitindo operações como inserção, busca, atualização, remoção e listagem dos elementos cadastrados. Em projetos desenvolvidos em C++, essa funcionalidade normalmente é implementada utilizando containers da STL, como `std::vector`, que oferecem gerenciamento eficiente das coleções de objetos.

De maneira geral, o vídeo demonstra que o projeto implementa corretamente as operações básicas de gerenciamento (CRUD), estabelece relacionamentos entre as entidades do sistema e aplica conceitos fundamentais da Programação Orientada a Objetos, como encapsulamento, organização em classes e interação entre objetos. A demonstração evidencia que o sistema atende aos requisitos propostos para o gerenciamento de projetos utilizando a metodologia Scrum, apresentando uma estrutura organizada e funcionalidades consistentes.

Diagrama a Nível de Módulos:

